

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

3-mashg'ulot. Pythonda TCP klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari yordamida client-server dasturini tuzish.
2. TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv).
3. Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close).

1-o'quv savol: Socket modulining asosiy metodlari yordamida client-server dasturini tuzish

Socket – tarmoq dasturlashda kompyuterlar o'rtasida ma'lumot almashish uchun ishlatiladigan asosiy vosita hisoblanadi. **Python**da socket moduli yordamida tarmoq orqali ma'lumot yuborish va olish mumkin. **TCP (Transmission Control Protocol)** protokoli yordamida server va client tizimlari o'rtasida ishonchli ulanish o'rnatiladi, ma'lumotlar paketlar orqali yuboriladi va qabul qilinadi.

Socket moduli yordamida quyidagi asosiy metodlar ishlatiladi:

`socket.socket()`: Soketni yaratish.

`connect()`: Serverga ulanish.

`send()`: Serverga ma'lumot yuborish.

`recv()`: Serverdan ma'lumot olish.

`bind()`: Serverni IP manzil va portga bog'lash.

`listen()`: Serverni ulanishlarni kutishga o'rnatish.

`accept()`: Mijozdan ulanishni qabul qilish.

`close()`: Soketni yopish.

Windows uchun TCP Client-Server dasturini tuzish:

1. Server Dasturi (Server kompyuterida ishlaydi):

Server kodi:

```
import socket

# Server soketini yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Serverni IP manzil va portga bog'lash
server_ip = "192.168.1.112" # Server kompyuterining IP manzili (bu yerda 192.168.1.10)
server_port = 5000 # Server porti (Client va Serverda bir xil bo'lishi kerak)

server_socket.bind((server_ip, server_port)) # Serverni IP va portga bog'lash
server_socket.listen(5) # Kutilayotgan ulanishlar soni
```

```

print(f"Server {server_ip}:{server_port} manzilida ulanishni kutmoqda...")

# Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish
data = conn.recv(1024)
print(f"Mijozdan olingan ma'lumot: {data.decode('utf-8')}")

# Mijozga javob yuborish
response = "Salom, klient! Men serverman."
conn.sendall(response.encode('utf-8'))

# Soketni yopish
conn.close()

```

Izohlar:

server_ip = "192.168.1.10": Bu serverning IP manzili. Server kompyuteri uchun real IP manzilini bilish kerak (boshqa kompyuterlar bu IP orqali serverga ulanadi). IP manzilini o'rganish uchun serverda `ipconfig` (Windows) yoki `ifconfig` (Linux/macOS) buyruqlarini bajarish mumkin.

bind(): Serverni IP manzil va portga bog'laydi.

listen(): Serverni ulanishlarni kutishga o'rnatadi.

accept(): Mijozdan ulanishni qabul qiladi.

recv(): Mijozdan ma'lumotni oladi.

sendall(): Mijozga javob yuboradi.

close(): Soketni yopadi.

2. Client Dasturi (Client kompyuterida ishlaydi):

Client kodi:

```

import socket

# Client soketini yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Serverga ulanish
server_ip = "197.181.1.112" # Server kompyuterining IP manzili (serverga ulanadi)
server_port = 5000 # Server porti (Server va Clientda bir xil bo'lishi kerak)

client_socket.connect((server_ip, server_port)) # Serverga ulanish

# Serverga ma'lumot yuborish
message = "Salom, server!"

```

```
client_socket.sendall(message.encode("utf-8"))

# Serverdan javob olish
response = client_socket.recv(1024)
print(f"Serverdan olingan javob: {response.decode('utf-8')}")

# Soketni yopish
client_socket.close()
```

Izohlar:

server_ip = "192.168.1.10": Bu yerda serverning IP manzili ko'rsatilgan, ya'ni server kompyuterining IP manzili. Bu manzilni **client** kompyuteriga to'g'ri ko'rsatish kerak (agar server kompyuteri boshqa tarmoqda bo'lsa, server kompyuterining real IP manzilini ishlatish zarur).

connect(): Serverga ulanishni amalga oshiradi.

sendall(): Serverga ma'lumot yuboradi.

recv(): Serverdan javobni oladi.

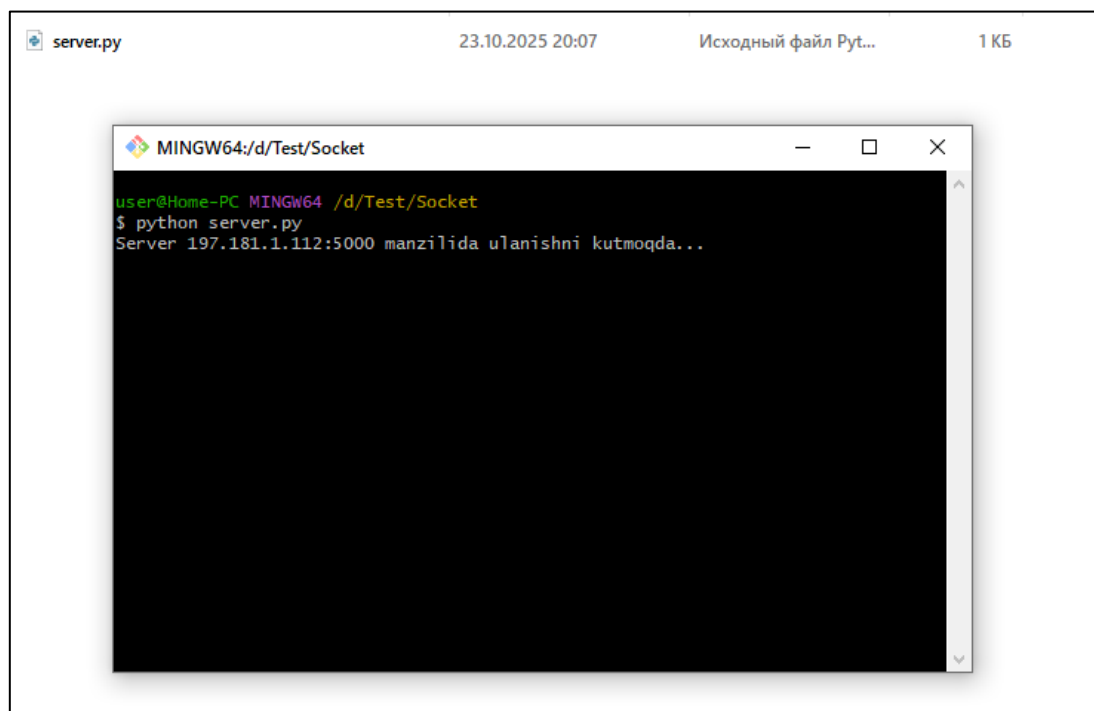
close(): Soketni yopadi.

Test qilish:

Serverni ishga tushirish:

Server dasturini server kompyuterida ishga tushurib, 192.168.1.10:5000 manzilida ulanishni kuting.

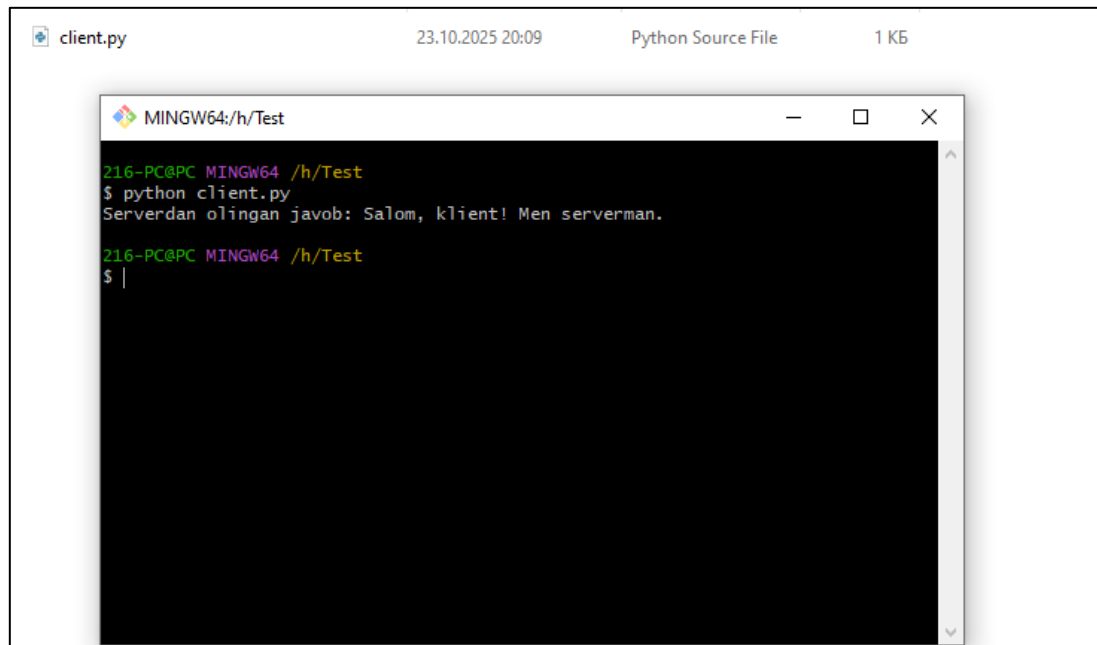
```
python server.py
```



Clientni ishga tushirish:

Client dasturini client kompyuterida ishga tushurib, serverga 192.168.1.10:5000 manzilida ulanishni amalga oshiradi.

python client.py



The screenshot shows a code editor window titled 'client.py' with a timestamp of '23.10.2025 20:09' and a file size of '1 KB'. Inside the editor is a terminal window titled 'MINGW64:/h/Test'. The terminal shows the following commands and output:

```
216-PC@PC MINGW64 /h/Test
$ python client.py
Serverdan olingan javob: Salom, klient! Men serverman.

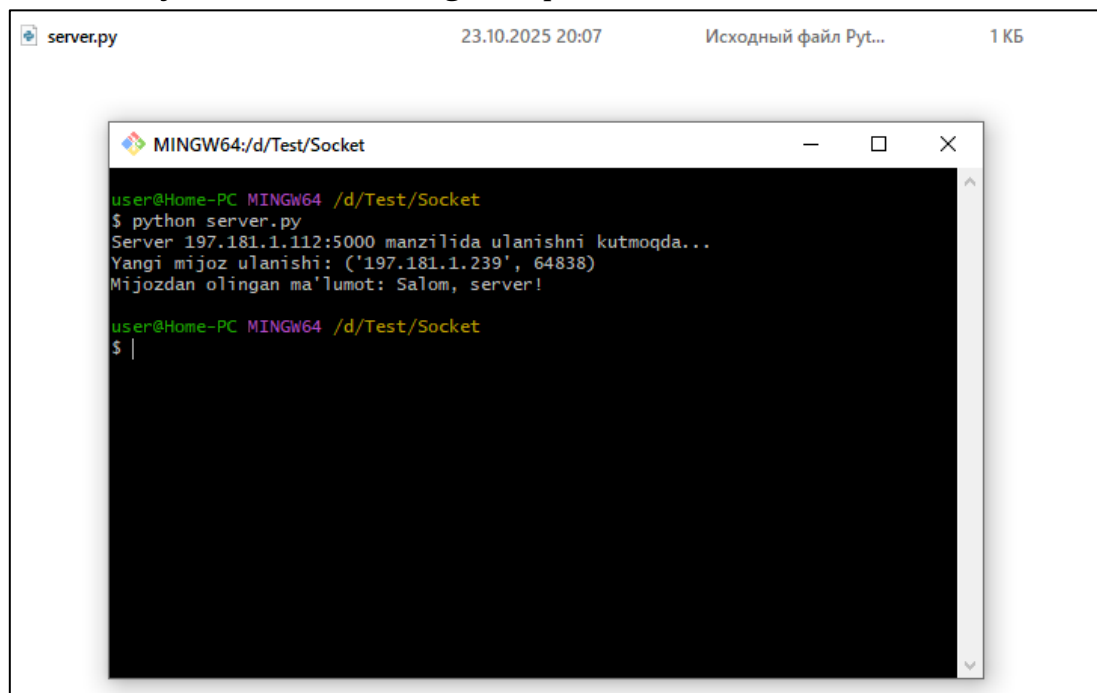
216-PC@PC MINGW64 /h/Test
$ |
```

Natija:

Client serverga "Salom, server!" xabarini yuboradi.

Server bu xabarni oladi va "Salom, klient! Men serverman." javobini yuboradi.

Client bu javobni olib, ekranga chiqaradi.



The screenshot shows a code editor window titled 'server.py' with a timestamp of '23.10.2025 20:07' and a file size of '1 KB'. Inside the editor is a terminal window titled 'MINGW64:/d/Test/Socket'. The terminal shows the following commands and output:

```
user@Home-PC MINGW64 /d/Test/Socket
$ python server.py
Server 197.181.1.112:5000 manzilida ulanishni kutmoqda...
Yangi mijoz ulanishi: ('197.181.1.239', 64838)
Mijozdan olingan ma'lumot: Salom, server!

user@Home-PC MINGW64 /d/Test/Socket
$ |
```

Xulosa:

Bu misolda, ikkita kompyuter yordamida **Python** va **socket** moduli yordamida TCP protokoli asosida server va client dasturlarini yaratdik. Server kompyuterida soket yaratilib, ulanishni kutmoqda, client esa serverga ulanishni amalga oshiradi, ma'lumot

yuboradi va javob oladi. Bu jarayon orqali tarmoq dasturlashning asosiy prinsiplari o'rganildi, jumladan, socketlar yordamida ma'lumot yuborish, qabul qilish.

2-o'quv savol: TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv)

TCP ulanishini o'rnatish:

TCP (Transmission Control Protocol) - tarmoqdagi ma'lumot uzatishning ishonchli protokoli bo'lib, u ma'lumotlarni paketlar shaklida uzatadi va o'zgartirishlar, xatoliklarni avtomatik tuzatadi. TCP orqali ulanish o'rnatishda, client (mijoz) serverga ulanadi va server ma'lumotni qabul qilib, javob qaytaradi.

Ulanishni o'rnatish:

Client tomonidan ulanishni o'rnatish:

`client_socket.connect((server_ip, server_port))`: Client dasturida serverga ulanishni boshlash uchun `connect()` metodi ishlatiladi. Bu metod serverning IP manzili va port raqami orqali ulanishni amalga oshiradi.

Server tomonidan ulanishni kutish:

`server_socket.listen(5)`: Server `listen()` metodi yordamida ulanishlarni kutadi. Bu metod, serverga keladigan ulanishlarni kutishga o'rnatadi.

`conn, addr = server_socket.accept()`: `accept()` metodi serverga kelgan ulanishni qabul qiladi va `conn` va `addr` yordamida mijoz bilan aloqani o'rnatadi.

Ma'lumot yuborish va olish:

`client_socket.sendall(message.encode("utf-8"))`: Client serverga ma'lumot yuboradi. `sendall()` metodi ma'lumotni serverga yuboradi. Yuboriladigan ma'lumot **bytes** formatida bo'lishi kerak, shuning uchun matnni `encode()` yordamida **bytes** ga o'zgartirish kerak.

`server_socket.recv(1024)`: Server `recv()` metodi yordamida clientdan ma'lumot oladi. Bu metod ma'lumotni **bytes** formatida qaytaradi.

`client_socket.recv(1024)`: Client serverdan javob olish uchun `recv()` metodini ishlatadi.

Tarmoqdagi xabar almashish jarayoni:

Client serverga ulanishni amalga oshiradi.

Client serverga ma'lumot yuboradi (masalan, "Salom, server!").

Server clientdan kelgan xabarni qabul qiladi va qayta ishlaydi.

Server clientga javob yuboradi (masalan, "Salom, klient! Men serverman.").

Client serverdan javobni oladi va ekranda chiqaradi.

Xulosa:

Bu o'quv savolida **TCP** ulanishi o'rnatish, ma'lumot yuborish va olish jarayonlari ko'rib chiqildi. `connect()`, `send()`, `recv()` metodlarining qanday

ishlashini o'rgandik. **Client-server** modeli orqali tarmoqda xabar almashishning asosiy printsiplari o'rganildi.

3-o'quv savol: Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close)

Xatoliklarni qayta ishlash:

Tarmoq dasturlarida ulanish xatoliklari tez-tez yuz berishi mumkin. **Socket** moduli bilan ishlashda xatoliklar yuzaga kelganda, bu holatni to'g'ri qayta ishlash zarur. **Python**da xatoliklarni **try-except** bloklari yordamida boshqarish mumkin. Xatolik yuz berishi mumkin bo'lgan kodlar **try** blokida joylashadi va xatolik yuzaga kelsa, ular **except** blokida ushlanadi.

Misol:

```
try:
    client_socket.connect((server_ip, server_port)) # Ulanishni amalga
    oshirish
except socket.error as e:
    print(f"Ulanishda xatolik yuz berdi: {e}") # Xatolikni qayta ishlash
```

Xatoliklarni qayta ishlash yordamida serverga ulanishda yuzaga keladigan barcha tarmoq muammolarini boshqarish mumkin, masalan:

Serverga ulanish o'rnatilmasa (masalan, server ishlamayapti yoki port band).

Tarmoq muammolari (masalan, internetga ulanmaslik).

Timeout o'rnatish:

Tarmoqda ishlayotgan dasturda **timeout** (vaqt chegarasi) xatoliklari yuzaga kelishi mumkin, ya'ni server yoki client bilan ulanishda uzoq vaqt kutish, foydalanuvchiga noma'lum holat yaratishi mumkin. **settimeout()** metodini ishlatib, socket ulanishi uchun vaqt chegarasini belgilash mumkin.

Misol:

```
client_socket.settimeout(10) # 10 soniya kutish
```

Agar 10 soniya ichida ulanish amalga oshmasa, xatolik yuz beradi. Bu holatni **try-except** blokida qayta ishlash mumkin.

Misol:

```
try:
    client_socket.connect((server_ip, server_port))
except socket.timeout:
    print("Ulanish uchun belgilangan vaqt tugadi.")
except socket.error as e:
    print(f"Ulanishda xatolik yuz berdi: {e}")
```

Reconnect qilish:

Agar ulanish uzilsa yoki dastur ishini to'xtatsa, **reconnect** funksiyasi yordamida ulanishni qayta o'rnatish mumkin. Bunda **try-except** blokidan foydalanish va tizimni qayta ulashni amalga oshirish kerak.

Misol:

```
def reconnect():
    try:
        client_socket.connect((server_ip, server_port))
        print("Serverga ulanish muvaffaqiyatli amalga oshirildi.")
    except socket.error as e:
        print(f"Ulanishda xatolik: {e}")
        reconnect() # Xatolik yuz bersa, qayta ulanishga harakat qilish
```

Agar ulanishda muammo bo'lsa, **reconnect()** funksiyasi yordamida tizim qayta ulanishga harakat qiladi. Bunda rekursiv ravishda **reconnect()** funksiyasi chaqiriladi.

Ulanishni yopish (close):

Ulanish tugagach, socketni yopish juda muhimdir. Agar socketni yopmasdan dasturdan chiqsa, tarmoq resurslari bo'shamaydi va boshqa ilovalar uchun band bo'lishi mumkin. **close()** metodi yordamida socketni to'g'ri yopish kerak.

Misol:

```
client_socket.close() # Socketni yopish
```

Serverda socketni yopish:

```
conn.close() # Serverga ulanishni yopish
```

Clientda socketni yopish:

```
client_socket.close() # Clientning ulanishini yopish
```

Xulosa:

Xatoliklarni qayta ishlash yordamida tarmoqda yuzaga keladigan xatoliklarni boshqarish, ulanishni tiklash va foydalanuvchiga to'g'ri javob qaytarish imkoniyati yaratiladi.

Timeout va **reconnect** funksiyalari yordamida ulanishlar o'rnatilishida muammolarni hal qilish va vaqt chegaralarini belgilash mumkin.

Socketni yopish (close) — tizim resurslarini to'g'ri boshqarish va ulanishni to'g'ri tugatish uchun zarur.

Bu o'quv savolida socket yordamida xatoliklarni boshqarish va tizimning uzilishlariga qarshi turish usullari ko'rib chiqildi.